Allyson Fuller
Student ID: 0763664

April 13, 2026

Interpreting the ASL (American Sign Language) Alphabet

INFO6147 - Deep Learning With Pytorch - Capstone Project

Dataset: https://www.kaggle.com/datasets/debashishsau/aslamerican-sign-language-aplhabet-dataset/data

GitHub: https://github.com/allysonmfuller/Interpreting-the-ASL-American-Sign-Language-Alphabet

Abstract

This project attempts to apply Deep Learning with Pytorch to an image dataset. The image dataset contained images of the American Sign Language (ASL) Alphabet, and was imported into our Python notebook via Kaggle API. Starting with 223,074 training samples across 29 classes, we subsampled 750 images per class to save time and computational recourses. Two baseline models were evaluated: ResNet18 and a custom CNN known as our ASL CNN. ResNet18 outperformed the ASL CNN and further hypertuning was performed on the baseline ResNet model. The final model, ResNEt50 trained for 25 epochs with a batch size of 30 and an Adam Optimizer achieved a final test accuracy of 78.06% and F1 score of 78.43%. These metrics indicate a model with strong generalization.

Introduction

Disability assistance is incredibly important and becoming easier and more accessible as technology evolves. For those who are deaf, hard of hearing, or experience speech disabilities, image (or video!) to speech recognition is important to bridge communication gaps in real time and fosters independence and dignity.

For many years, those who are hard of hearing have relied on written word (writing, typing, drawing) to communicate. In television, for example, the world went from silent films which were accessible to most, to talking films and foreign films with subtitles. Closed captioning was later introduced, but both subtitles and closed captioning required an individual to transform scripts or follow the video and transcribe in real time. By the 1990's "an embedded chip was even invented ... [allowing] televisions themselves decode captions for viewers" (https://www.rev.com/blog/history-of-closed-captions).

Now there is Artificial intelligence that can generate speech to text or even image to text or speech. For the latter, the question was: why is it important for those who are deaf, hard

of hearing, or experience speech disabilities to have image or video to text or speech tools? Why can they not just simply type or write what they want?

American Sign Language (ASL) is a language of it's own. For some deaf individuals, their first language is considered ASL and their second is considered English. Therefore, that image transcription is important to capture the nuances of ASL. Image to text or speech allows communication that is faster, more accurate, and can capture tone or emotion. These factors are even more important in emergency situations.

There are datasets containing entire ASL words; for this report we will be focusing solely on the ASL alphabet. Using the "Interpreting ASL Alphabet" dataset sourced from Kaggle, the objective is to be able to predict from which class (A-Z) a photo of a hand may belong.

Methodology

Data Selection

The dataset chosen is the ASL (American Sign Language) Alphabet, which contains a training set and a testing set. The Training Set consists of 29 classes (A-Z, space, delete, nothing) with a total of 223,074 images while the Test Set contains only 29 images.

Due to the size of the dataset, the retrieval was performed through an API code:

```
import json

kaggle_dict = {
    "username": "allysonofagun",
    "key": "KGAT_de2653a318fb05f8040e21c007fc6a7b"
}

with open("kaggle.json", "w") as f:
    json.dump(kaggle_dict, f)

!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

!kaggle datasets download -d debashishsau/aslamerican-sign-language-aplhabet-dataset

# Unzip
!unzip -q aslamerican-sign-language-aplhabet-dataset.zip -d asl_dataset
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.0), please consider upgrading to the latest version (2.0.1)
Dataset URL: https://www.kaggle.com/datasets/debashishsau/aslamerican-sign-language-aplhabet-dataset
License(s): CC0-1.0
Downloading aslamerican-sign-language-aplhabet-dataset.zip to /content
100% 4.20G/4.20G [00:21<00:00, 205MB/s]
```

Preprocessing

First, a few random images were selected for viewing:

With 223,074 samples of the training data and only 29 in the test data, it was decided to use the training set for training and testing by keeping all 29 classes, with 750 images per class.

Additional data preprocessing steps included

- resizing images the images to 224 x 224
- normalizing pixel values using ImageNet mean and standard deviation (https://www.kaggle.com/datasets/puneet6060/intel-image-classification)

- applying data augmentation techniques to TRAINING ONLY, such as randomly:
  - flipping the image left-to-right
  - rotating the image by random angles
  - adjust the brightness and contrast of an image

The dataset was split into Training (500 images) and Testing (250 images), and Training was further split into Training (80%) and Validation (20%)

Model Selection and Architecture

For image classification, there is a pre-trained model known as ResNet, which is a "residual learning framework" (https://arxiv.org/abs/1512.03385) using CNN architecture and trained on the ImageNet dataset which "spans 1000 object classes and contains 1,281,167 training images, 50,000 validation images and 100,000 test images" (https://www.image-net.org/download.php)

For model selection, it felt proper to compare the pre-trained ResNet model to a custom-made CNN Model. Below is a table outlining the Architecture of ResNet 18 and the ASL CNN created.

Architecture	*ResNet 18*	*ASL CNN*
ACCURACY	**74%**	
Number of Layers	18	14
Convolutional Layers	☑	☑
Fully Connected Layers	1	3
Filter Depth	64 → 64 → 128 → 128 → 256 → 256 → 512 → 512	32 → 64 → 128 → 256
Skip Connections	☑	☒
Pooling	1 early MaxPool Global Ave Pool at end	MaxPool after every block
ReLU	☑	☑
Batch Normalization	☑	☑
Dropout	☒	☑
Optimizer	Adam	Adam
Batch Normalization	☑	☑
Starting Weights	Pre-Trained	Random

Skip connections are known to be a defining factor of ResNet. They perform by allowing gradients to flow backward more cleaning during training.

For the ASL CNN created:

- it was separated into blocks because each block doubles the filter count and halves spatial dimension with MaxPool. More filters usually result in richer features as spatial detail is compressed.
- Batch normalization was introduced after every convolutional layer and before the ReLU layer because it works by reducing sensitivity to the initial random weights. It normalizes the convolutional layer so it does not get too big or too small.
- ReLU introduces non-linearity and avoids the vanishing-gradient problem by setting negative values to 0.
- MaxPool halves the image dimensions after every CNN block. It does this by taking the maximum value in every 2x2 window. ResNet does the same, but just once then uses strided convolutions
- After every convolutional block we have the tensor shape of (256, 14, 14) and this is flattened in the CNN. ResNet does NOT do this because it averages each channel down to one number first.
- Dropout randomly zeros out neurons and this is done twice in the CNN where it zeroes out 50% then 30% so the neural network does not memorize or rely on one path. ResNet does not need this because of the batch normalization and skip connections
- No SoftMax because PyTorch's “nn.CrossEntropyLoss” applies “log_softmax”, so the model correctly returns raw logits (https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html)

Model Training

Model training was performed on both the ResNet Model and ASL CNN model to see which performed better. Parameters were kept pretty basic in preparation for the next hypertuning step, but were the exact same for both the ResNet Model and ASL CNN model for proper comparison.

In the end, the validation accuracy for the ResNet Model was over 70% while the ASL CNN model validation accuracy was incredibly low at 3%. The ResNet model was also more fast and less computationally expensive. Therefore, hyperparameter training was performed on the ResNet Model.

Hyperparameter Training

To compare parameters properly, one parameter was changed at a time to view the overall effect on the model.

The parameters changed on the base ResNet model included:

Evaluation

For each change in hyperparameter, metrics captured included:

- Final train Loss
- Final train Accuracy
- Final validation Loss
- Final validation Accuracy
- Accuracy
- Precisions
- Recall
- F1 Score

A table was then created to compare all metrics across parameters:

Model Version	*Train Loss*	*Train Acc (%)*	*Val Loss*	*Val Acc (%)*	*Accuracy (%)*	*Precision (%)*	*Recall (%)*	*F1 (%)*
ResNet Baseline	**0.90**	**74.00**	**0.88**	**74.17**	**61.50**	**41.04**	**33.85**	**35.35**
CNN Baseline	3.39	3.49	3.38	3.28	1.39	3.79	3.68	1.01
ResNet34	0.86	75.04	0.86	75.24	69.00	44.03	37.50	39.82
SGD	2.45	46.92	2.41	47.48	19.50	15.29	10.18	10.83
Increased Epochs, 10 -> 20	0.77	76.89	0.81	77.00	69.50	38.00	35.56	35.54
Increased Batch Size, 32 -> 50	0.91	74.28	0.93	73.28	69.80	60.96	63.05	60.08
Increased LR, 0.001 - > 0.01	1.47	68.49	1.42	69.59	62.50	40.73	36.52	37.84

From this table we can deduce overall the ALS CNN model performed abysmally. Other obvious take aways include that the optimizer Adam performed better than SGD.

Changing the model to one with deeper layers, ResNet34, and increasing epochs, and increasing learning rate all helped the model to perform better. In addition, increase batch sized allowed the model to perform much better. Therefore these are parameters focused on during fine-tuning.

We can see these results reflected visually as well:

Fine-Tuning and Iteration

As previously mentioned, increasing layer depth, changing epochs, and changing batch numbers resulted in a better performance in the model.

Technique	*Config Value*	*Train Loss*	*Train Accuracy*	*Val Loss*	*Val Accuracy*	*Accuracy*	*Precision*	*Recall*	*F1*	*Training Time (min)*
Model Depth	resnet34	0.87	74.72	0.85	74.52	74.52	76.51	74.32	74.35	11.19
	resnet50	0.81	75.50	0.80	76.34	76.34	78.70	76.50	76.39	12.29
Epochs	25	0.72	77.95	0.74	77.41	77.41	78.77	77.06	77.29	28.04
	30	0.69	78.79	0.75	77.59	78.59	79.37	77.78	77.78	33.57
Batch Size	10	0.98	70.20	0.86	73.76	73.76	77.45	73.70	73.70	13.29
	20	0.88	73.55	0.85	74.10	74.10	76.26	73.98	73.82	11.75
	30	0.86	75.27	0.89	74.38	74.38	76.40	74.45	74.40	11.22
	40	0.86	75.16	0.86	75.10	75.10	76.15	75.04	74.87	11.18

From the results above, 30 epochs is clearly the winner however it is much more computationally and time expensive. We can trade off to 25 epochs which just performed slightly less well, and we will choose a batch size of 30. Lastly ResNet50 performed better, so we will choose this hyperparameter as well.

Final Model Testing

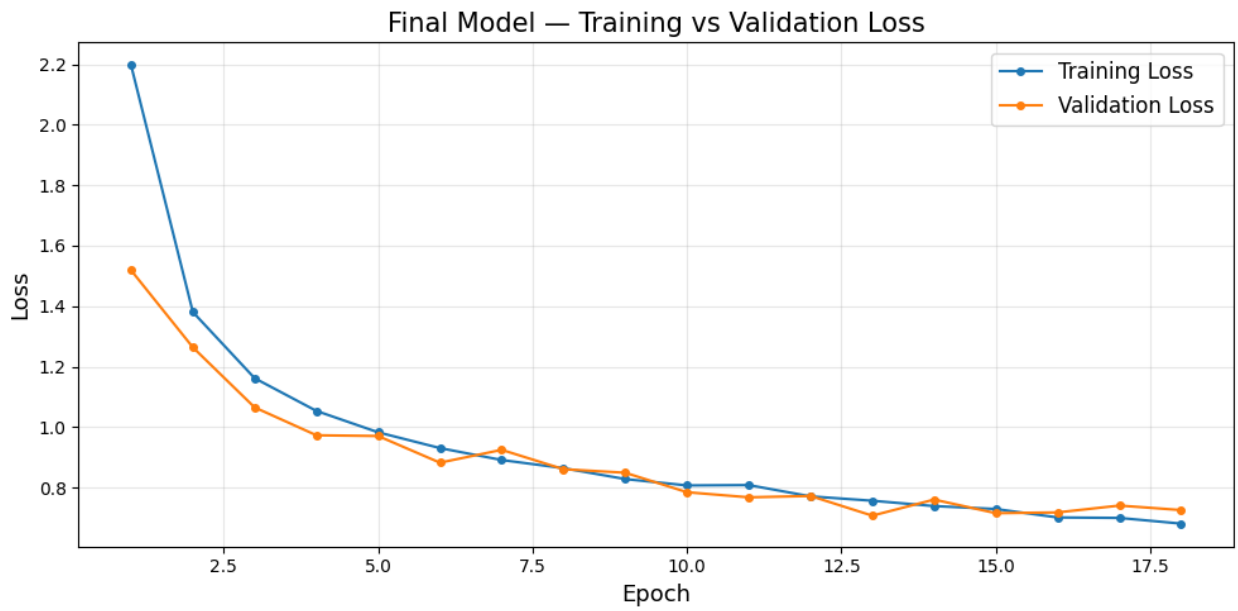
Final model testing included choosing 25 epochs but introducing Early Stopping to save time and computation. We landed on a batch size of 30 and using the ResNet50 model.

Training and validation were performed again on this final model. We can visualize the Loss below:

The Final Model Metrics are as follows:

- Accuracy: 74.18%
- Precision: 74.14%
- Recall: 74.08%
- F1: 74.00%

We can also visualize the Loss below:



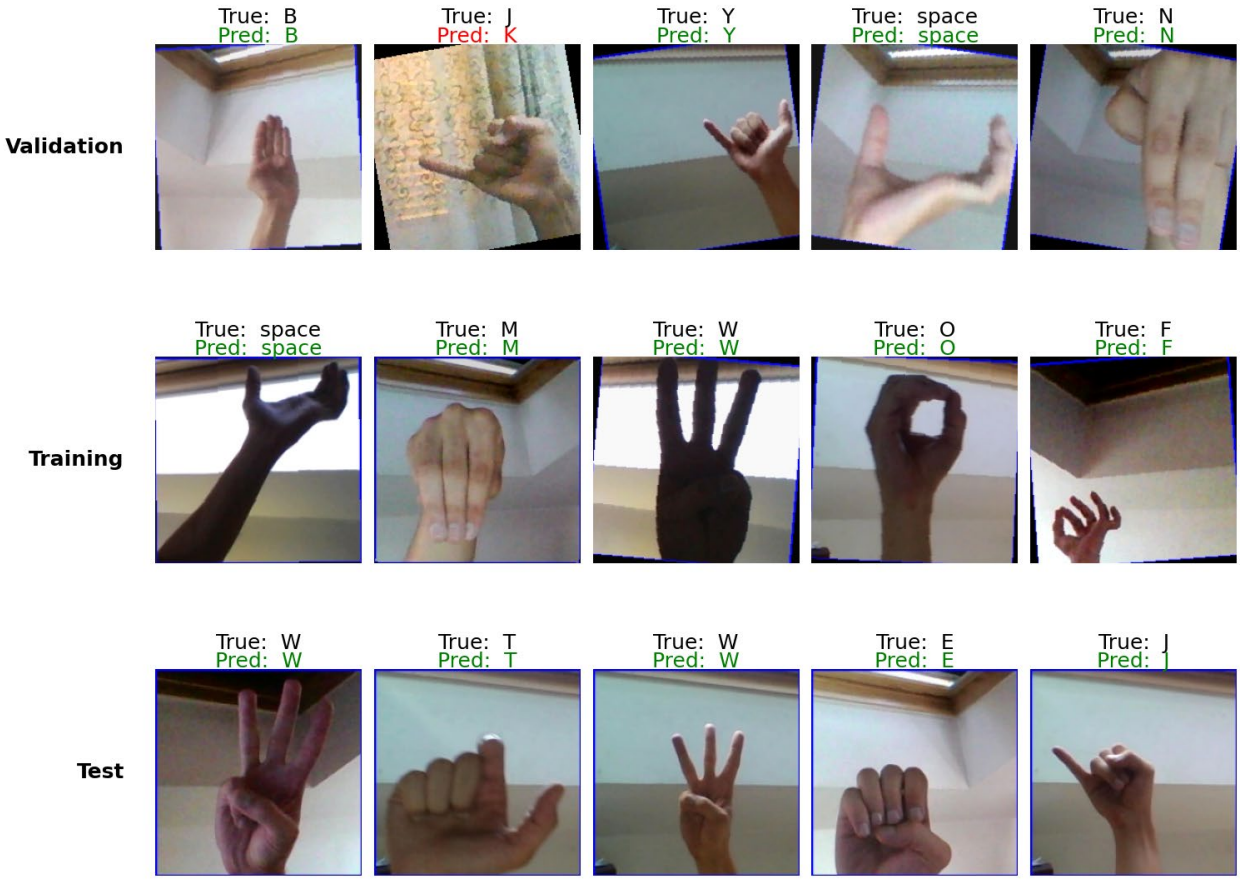
Results

The model was then applied to the Test set, with the results:

- Accuracy: 78.06%
- Precision: 80.98%
- Recall: 78.06%
- F1: 78.43%

These results indicate a strong predictive model. Specifically looking at the F1 score which is the harmony between Precision and Recall, we can say that our of 100 images, our model can correctly guess 78.

Final Model — Training, Validation and Test Predictions



Conclusion

Overall, we found that the pre-trained ResNet models outperformed the CNN model by a landslide. Moving forward with the ResNet model, we tuned hyperparameters such as model depth by trying out ResNet34 instead of ResNet18, optimizers, number of epochs, batch sizes and learning rate. From there we learnt ResNet, epochs and batches had the most influence so we performed some experimentation with various values for each parameter.

Our final Training, Validation and Testing performed very well with a final testing F1 score of 78.43%. Our Training and Validation Loss converged nicely, and the accuracy scores were all very close indicating the model was generalizing well without over or underfitting.